SourceAgent Source Detection Multi-Layer Update Specification

Filter → Candidate → Classifier → Verifier
(with optional LLM-assisted semantic recognition)

Version: 1.0
Date: 2026-03-03

Document Map

This document defines a layered architecture for SourceAgent's source detection module, including:

- Layer responsibilities and data contracts (inputs/outputs).
- An end-to-end pipeline structure: Filter → Candidate Miners → Semantic Classifier (Rules + optional LLM) → Verifier.
- A verifier-first, fail-closed policy for final labels.
- A complete implementation prompt (English) for an assistant model (e.g., Claude) to generate code changes.
- A structure diagram and recommended test/evaluation plan.

1. Overview

SourceAgent aims to recover high-level, security-relevant input sources from embedded firmware binaries (ARM Cortex-M, ELF and stripped/raw .bin). The project currently supports multiple source labels, such as:

- MMIO_READ
- ISR_MMIO_READ
- ISR_FILLED_BUFFER
- DMA_BACKED_BUFFER

This update proposal refactors the source detection logic into a multi-layer pipeline with explicit contracts and a strict verifier-first policy. The layered structure makes it easier to:

- Keep mining deterministic and scalable (avoid full emulation).
- Use optional LLM assistance only where it is safe and verifiable (semantic recognition), without letting it directly assert final results.
- Separate address recovery (register-level) from semantic source labeling (label-level) in evaluation.
- Reduce metric confusion (candidates vs verified labels) and enable stronger regression testing.

1.1 Goals

- Define a Filter → Candidate → Classifier → Verifier architecture for source detection.
- Specify each layer's tasks, inputs, outputs, invariants, and failure behavior.
- Integrate existing address recovery improvements (e.g., typed struct field access recovery) as an upstream signal for source mining.
- Allow an optional LLM-driven semantic classifier that produces JSON-only structured outputs that must be verified.
- Provide an implementation prompt for a code-generation assistant to implement the architecture and update evaluation accordingly.

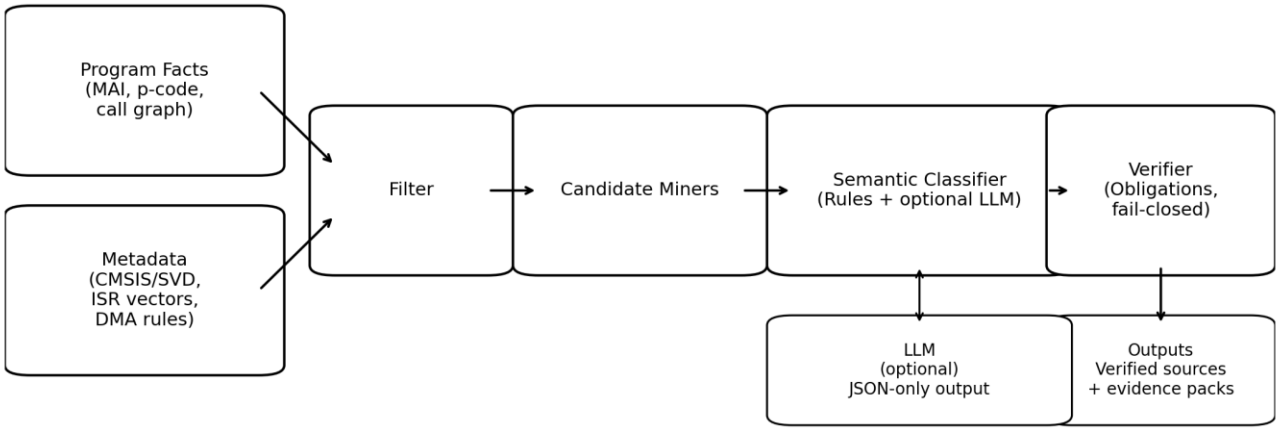
1.2 Non-goals

- Not attempting full dynamic analysis, full-system emulation, or execution rehosting.
- Not allowing any LLM output to bypass static verification.
- Not requiring perfect type recovery; non-HAL firmware should still benefit from IR-based heuristics and bounded slicing.

2. Architecture Summary

The updated module is a pipeline of four layers. Each layer produces structured artifacts that are consumed by downstream layers. Only the Verifier layer is allowed to emit final labels.

Figure 1. High-level pipeline and optional LLM callout.



At a high level: the Filter layer selects relevant program facts (e.g., MMIO reads, DMA configuration writes). Candidate Miners propose label hypotheses and create evidence packs. The Semantic Classifier refines hypotheses (rules-first, optionally LLM-assisted) and emits structured classification proposals. The Verifier checks each proposal against static obligations and returns VERIFIED/PARTIAL/REJECTED results.

3. Shared Data Model

All layers communicate using a shared set of data structures. The exact implementation can use Python dataclasses or typed dictionaries. The fields below are intentionally explicit to support deterministic testing and caching.

3.1 Core entities

Entity	Purpose	Key fields (examples)

AccessFact	A normalized memory access fact extracted from IR/p-code and/or decompiler patterns.	`site_addr (int)`
		`function_addr (int)`
		`direction (READ|WRITE)`
		`addr_expr (AddrExpr)`
		`value_expr (Expr)`
		`provenance (enum)`
		`evidence (list[EvidenceItem])`

AddrExpr	Symbolic representation of an address; may be a constant, base+offset, dereference chain, or unknown.	`kind (CONST|BASE_PLUS_OFF|DEREF|UNKNOWN)`
		`const (optional int)`
		`base (optional Expr)`
		`offset (optional int)`
		`constraints (optional)`

Candidate	A hypothesis generated by a miner for a specific label, before semantic classification.	`candidate_id (str)`
		`label_family (SOURCE)`
		`site (SiteRef)`
		`primary_fact_ids (list[str])`
		`hypothesis (dict)`
		`evidence_pack (EvidencePack)`
		`obligations (list[Obligation])`

Classification	A classifier output: refined label + parameters, with justification. May come from rules or LLM (but must be verifiable).	`candidate_id (str)`
		`proposed_label (enum)`
		`params (dict)`
		`confidence (float 0..1)`
		`rationale (str)`
		`additional_obligations (list[Obligation])`
		`classifier_meta (dict)`

Verdict	Verifier output for a classified candidate.	`candidate_id (str)`
		`status (VERIFIED|PARTIAL|REJECTED)`
		`verified_label (optional VerifiedLabel)`
		`failed_obligations (list[str])`
		`verifier_meta (dict)`

VerifiedLabel	Final, fail-closed output used by downstream components and evaluation.	`label (enum)`
		`site (SiteRef)`
		`io_addr (optional int)`
		`buffer (optional BufferRef)`
		`context (ISR|THREAD|DMA)`
		`evidence_pack_id (str)`
		`confidence (optional)`

EvidencePack	Bundle of evidence items enabling reproducible verification and optional semantic classification.	`pack_id (str)`
		`facts (list[EvidenceItem])`
		`snippets (optional)`
		`notes (optional)`
		`artifacts (paths/hashes)`

Obligation	A verifier-checkable predicate that must hold for a candidate/classification to be VERIFIED.	`obligation_id (str)`
		`type (enum)`
		`inputs (dict)`
		`severity (MUST|SHOULD)`
		`explain (str)`

4. Layer Specifications

This section specifies the responsibilities and contracts of each layer. Each layer MUST treat upstream artifacts as immutable inputs and produce deterministic outputs for the same inputs and configuration.

4.1 Filter Layer

Select and normalize the subset of facts relevant to source detection. The Filter does not decide labels; it prepares high-signal inputs for miners and reduces noise.

Item	Details
Primary tasks	• Normalize all memory accesses into AccessFact records with explicit direction (READ/WRITE). • Apply address-range filters (e.g., MMIO region, optional system peripheral handling). • Derive higher-level 'events' such as MMIO_READ_EVENT or DMA_CONFIG_EVENT. • Attach minimal evidence: p-code ops, instruction bytes, decompiler snippet pointers.
Input	• Program IR facts: MemoryAccessIndex / p-code-derived load/store facts. • Optional: typed struct field access facts (STRUCT_FIELD/STRUCT_RESOLVED). • Metadata: MCU memory map, peripheral base ranges, ISR vector table, optional SVD/CMSIS offsets. • Configuration knobs (see Section 6).
Output	• FilteredAccessFacts: list[AccessFact] • DerivedEvents: list[Event] (e.g., mmio_reads, dma_configs) • FilterReport: counts by category, dropped reasons, optional debug sampling.
Hard invariants	• Never fabricate concrete io_addr values. If unknown, keep AddrExpr as symbolic/UNKNOWN. • Must preserve direction correctly (a read must not become a write). • Must be deterministic (no randomness).

4.2 Candidate Miners Layer

Generate label hypotheses as Candidates. Miners should be simple and composable; their job is to propose, not to prove.

Item	Details
Primary tasks	• Enumerate candidate sources for each label family using Filter outputs. • Build evidence packs sufficient for downstream classification and verification. • Emit verifier obligations (predicates) rather than making unverifiable assumptions.
Input	• FilteredAccessFacts and DerivedEvents from Filter layer. • Interprocedural binding results (if available): e.g., instance binding for HAL handle->Instance. • Auxiliary analyses: bounded def-use slices, call graph, basic block CFG, ISR handler set.
Output	• Candidates: list[Candidate], each with hypothesis + evidence_pack + obligations. • MinerReport: per-label counts, top failure modes, coverage of peripheral categories.
Hard invariants	• Candidates can be over-approximate, but must be explainable and verifiable. • All candidate fields must be serializable (for caching and eval).

Recommended miners (initial)	• MMIO_READ miner: READ AccessFacts in thread context. • ISR_MMIO_READ miner: READ AccessFacts where function is an ISR handler. • ISR_FILLED_BUFFER miner: ISR MMIO reads whose value flows to a memory write within a bounded slice. • DMA_BACKED_BUFFER miner: DMA config patterns indicating peripheral-to-memory transfers; output buffer sources.

4.3 Semantic Classifier Layer (Rules + optional LLM)

Refine each Candidate into a structured Classification proposal: exact label + params + rationale. This layer may use deterministic rules and optionally an LLM, but must output strict JSON and remain verifier-friendly.

Item	Details
Primary tasks	• Run rules-first classification (cheap, deterministic). • Optionally call an LLM only when rules are uncertain or ambiguous. • Produce a structured classification output (JSON) following a strict schema. • Add obligations for verifier when the classification relies on contextual assumptions.
Input	• Candidate with evidence_pack and obligations. • Optional: decompiler snippets or summarized IR traces inside evidence_pack. • LLM configuration from .env (provider, model, key, temperature, timeout).

Output	
	• Classification: proposed_label + params (io_addr, buffer, context) + rationale + confidence. • Additional obligations for verifier (e.g., require confirming dataflow path). • ClassifierReport: LLM call counts, cache hit rate, uncertainty stats.
Hard invariants	
	• Classifier must never mark anything final; only propose. • LLM output must be JSON-only and validated against schema; otherwise fall back to rules-only classification. • No non-deterministic changes without explicit configuration (e.g., temperature must be low or 0 for reproducibility).

4.4 Verifier Layer

Prove or reject proposed labels. The verifier is the only layer allowed to emit final outputs.

Item	Details
Primary tasks	
	• Check obligations using static program facts (p-code, def-use, memory maps, interprocedural bindings). • Resolve symbolic AddrExpr to concrete addresses only when provable. • Emit VERIFIED/PARTIAL/REJECTED verdicts. • Package final evidence packs (including proof artifacts) for auditing.
Input	
	• Classification proposals + Candidate obligations + program facts. • Auxiliary metadata (MCU maps, SVD/CMSIS offsets).

Output

- Verdicts: list[Verdict]
- VerifiedLabels: list[VerifiedLabel] (only from VERIFIED verdicts; PARTIAL may be stored separately).
- VerifierReport: obligation failure breakdown, unresolved addr_expr stats.

Hard invariants

- Fail-closed: if an obligation cannot be proven, output PARTIAL or REJECTED.
- Never 'guess' io_addr. Unknown remains unknown.
- Every VERIFIED label must have a minimal, reproducible evidence pack.

5. Optional LLM-Assisted Semantic Recognition

The Semantic Classifier may optionally call an LLM to resolve ambiguous cases, such as distinguishing an ISR-filled buffer vs a one-off MMIO read with no buffering, or interpreting library wrappers where deterministic rules are weak. LLM assistance must be bounded and verifier-friendly.

5.1 When to call the LLM

- Only after deterministic rule-based classification yields an 'UNCERTAIN' outcome.
- Only when the evidence pack contains enough context (e.g., decompiler snippet + short def-use trace).
- Never call the LLM for trivial cases already covered by rules (for cost, speed, and reproducibility).

5.2 LLM I/O contract

The LLM MUST return JSON only. The JSON is validated against a schema before being accepted. If validation fails, the classifier falls back to rules-only output.

```
{
  "candidate_id": "string",
  "proposed_label": "MMIO_READ | ISR_MMIO_READ | ISR_FILLED_BUFFER | DMA_BACKED_BUFFER | UNKNOWN",
  "confidence": 0.0,
  "params": {
    "io_addr": "optional int (hex string allowed)",
    "buffer": {
      "kind": "GLOBAL|STACK|HEAP|UNKNOWN",
      "addr": "optional int",
      "size": "optional int"
    },
    "context": "ISR|THREAD|DMA|UNKNOWN"
  },
  "rationale": "short text",
  "required_obligations": [
    {"type": "DATAFLOW_EXISTS", "inputs": {...}},
```

```
    {"type": "ISR_CONTEXT", "inputs": {...}}
  ]
}
```

5.3 Suggested .env settings

This spec assumes an existing .env-based configuration system. The exact variable names may differ in your repository; the following are recommended conventions.

Variable	Example	Purpose
LLM_ENABLE	`true|false`	Enable the LLM-assisted classifier step.
LLM_PROVIDER	`openai|anthropic|...`	Backend provider implementation to use.
LLM_MODEL	`string`	Model name (e.g., gpt-5, claude-3.5...).
LLM_API_KEY	`string`	Secret key (never log this).
LLM_TEMPERATURE	`0.0`	Use 0 (or near 0) for deterministic outputs.
LLM_MAX_TOKENS	`integer`	Hard cap for completion length.
LLM_TIMEOUT_SECS	`integer`	Timeout per request; must fail fast to rules-only.
LLM_CACHE_DIR	`path`	Cache JSON outputs keyed by candidate+evidence hash for reproducibility.
LLM_STRICT_JSON	`true`	Reject non-JSON output; no auto-repair in production runs.

5.4 Safety and reproducibility rules

- Log only hashes of prompts/evidence and the JSON output; avoid logging raw secrets or entire binaries by default.
- Cache LLM outputs by (candidate_id, evidence_pack_hash, model, temperature) so reruns are stable.
- Treat LLM output as untrusted input: validate JSON schema and sanitize strings before storage.
- Verifier is authoritative: LLM cannot bypass obligations.

6. Update Roadmap (Recommended)

The architecture can be implemented incrementally. The following roadmap prioritizes correctness and evaluation clarity.

6.1 P0: Contracts + evaluation correctness

1. Introduce explicit data structures (AccessFact, Candidate, Classification, Verdict, VerifiedLabel).
2. Ensure MemoryAccessIndex emits direction (READ/WRITE) for every access, including struct-field-derived accesses.
3. Update evaluation harness to separate: (a) MMIO address recovery, and (b) source label recovery (only VERIFIED labels).
4. Plumb STRUCT_FIELD/STRUCT_RESOLVED signals into the interprocedural resolver instead of expanding to all same-type bases.

6.2 P1: Implement miners and verifier obligations per label

5. MMIO_READ and ISR_MMIO_READ miners + verifier checks (address range, read semantics, ISR context).
6. ISR_FILLED_BUFFER miner using bounded def-use slice from read value to a memory write; verifier proves the slice path.
7. DMA_BACKED_BUFFER miner that recognizes DMA configuration (peripheral-to-memory) and produces buffer sources; verifier proves the configuration chain.
8. Add rules-first classifier + optional LLM fallback for ambiguous cases.

6.3 P2: Generalization and scaling

9. Replace hand-written offset tables with CMSIS/SVD-derived offset maps; add coverage checks (USB/CAN etc.).
10. Add non-HAL heuristics: base+offset normalization for RIOT/NuttX patterns without type recovery.
11. Add caching, sampling, and debug report tooling so large firmware runs remain inspectable.

7. Acceptance Criteria and Tests

The update should be accompanied by explicit unit tests and end-to-end regression tests. The goal is not only better metrics, but stable, explainable behavior.

7.1 Required tests

- Unit tests for Filter layer: correct direction tagging, address-range filtering, and deterministic outputs.
- Unit tests for each miner: produce expected Candidates for microbench binaries (T0).
- Verifier tests: each obligation type has at least one positive and one negative case.
- LLM classifier tests: schema validation, fallback behavior, caching stability.

7.2 Required regressions

- Paired-build regression: microbench ELF vs raw .bin should produce consistent VERIFIED labels (within expected mapping).
- P2IM STM32F103 suite: report both address-recovery metrics and label-recovery metrics; keep historical baselines.

Appendix A. Complete Update Prompt (English)

Copy/paste the prompt below into your implementation assistant (e.g., Claude). It is written to request concrete code changes, data contracts, and tests.

```
You are an expert static-analysis engineer. Help me refactor and extend the SourceAgent "source
detection" module into a verifier-first, multi-layer pipeline.

CONTEXT
- Domain: ARM Cortex-M embedded firmware analysis (ELF and stripped/raw .bin).
- Objective: recover security-relevant "input sources" with labels:
  1) MMIO_READ
  2) ISR_MMIO_READ
  3) ISR_FILLED_BUFFER
  4) DMA_BACKED_BUFFER
- Existing pieces (names may vary):
  - MemoryAccessIndex (MAI) extracts load/store facts from Ghidra IR/p-code and some decompiler
pattern matching.
  - There is already an interprocedural stage (stage 2.5) to propagate/bind facts across calls.
  - A verifier exists; outputs must be VERIFIED, otherwise PARTIAL/REJECTED.
- A recent update added typed struct-field recovery for HAL patterns like handle->Instance->FIELD.
This improves "MMIO address recovery" but is not yet cleanly integrated into semantic source labeling.

PROBLEM TO SOLVE
MMIO address recovery and semantic source labeling are currently mixed in both code paths and
evaluation. I need a clean architecture:
  Filter -> Candidate Miners -> Semantic Classifier (Rules + optional LLM) -> Verifier
where ONLY the verifier can emit final labels.

HARD CONSTRAINTS
- Deterministic + scalable. No full-system emulation.
- Avoid brittle decompiler text where possible; prefer p-code/def-use facts.
- LLM is OPTIONAL and can ONLY operate in the semantic classifier layer. It must output strict JSON
and never bypass verification.
- Fail-closed: never guess MMIO addresses or labels.

YOUR TASKS
1) Architecture + data contracts
- Define explicit data structures: AccessFact, Candidate, EvidencePack, Obligation, Classification,
Verdict, VerifiedLabel.
- For each layer define: tasks, input types, output types, invariants, and failure behavior.

2) Refactor into four layers
A) FILTER
- Input: MAI access facts (+ metadata: MCU map, ISR vectors, DMA rules, optional SVD/CMSIS offsets).
- Output: FilteredAccessFacts + DerivedEvents (mmio_reads, dma_configs, isr_contexts).
- MUST: tag direction (READ/WRITE) for every access.
- MUST NOT: fabricate concrete io_addr if it is not provable.

B) CANDIDATE MINERS
- Implement miners that generate Candidates:
  * MMIO_READ: read events in thread context
  * ISR_MMIO_READ: read events inside ISR handler
  * ISR_FILLED_BUFFER: ISR read whose value flows to a memory write (bounded slice)
  * DMA_BACKED_BUFFER: DMA config indicates peripheral-to-memory transfer to a buffer
- Each Candidate includes an EvidencePack and a list of verifier Obligations.

C) SEMANTIC CLASSIFIER (Rules-first, optional LLM)
- Input: Candidate (+ evidence pack).
- Output: Classification with proposed_label + params + confidence + rationale + any extra
obligations.
- Rules-first: try deterministic rules; only call LLM if uncertain.
- LLM requirements:
  * JSON-only output; validate against schema
```

```
  * temperature=0 (or near 0) for reproducibility
  * cache outputs by (candidate_id + evidence_hash + model settings)
  * if JSON invalid or timed out -> fall back to rules-only classification

D) VERIFIER
- Input: Classification + Candidate obligations + program facts
- Output: Verdict + VerifiedLabel (only if VERIFIED)
- Implement obligation types such as:
  * ADDR_IN_MMIO_RANGE
  * IS_READ_SEMANTICS
  * ISR_CONTEXT
  * DATAFLOW_EXISTS (read value reaches buffer write)
  * DMA_CONFIG_VALID (periph->mem, periph addr, mem addr, length)
- Fail-closed: if not provable -> PARTIAL/REJECTED

3) Integrate typed struct-field recovery into the mainline (do NOT over-expand)
- Instead of expanding handle->Instance->FIELD to "all base addresses of that TypeDef", produce a
symbolic AddrExpr:
    DEREF(handle + instance_off) + field_off
  and let interprocedural binding + verifier resolve it to a concrete io_addr when provable.
- If binding cannot be proven, keep it PARTIAL (with evidence).

4) Update evaluation
- Report two independent metrics:
  (a) MMIO address recovery (register-level)
  (b) Source label recovery (label-level; ONLY VERIFIED labels)
- For label-level metrics, align to "read semantics": do not count pure write-only registers as
MMIO_READ sources.

5) Tests
- Layer unit tests: filter correctness, miner output, classifier schema validation, verifier
obligation checks.
- End-to-end regressions:
  * microbench T0: ELF and raw .bin paired consistency
  * P2IM F103 suite: baseline metrics + improved metrics

DELIVERABLES
- Patch plan (P0/P1/P2) and expected impact.
- Proposed file/module layout and minimal code skeletons (Python).
- Exact JSON schema for LLM output and the prompting template.
- Acceptance checklist and debugging playbook (logs, reports, sampling).

IMPORTANT: Provide implementation details, not just prose. Give pseudocode or minimal function
signatures for each layer. Ensure the design is verifier-friendly and fail-closed.
```